

Practice Sheet: this, call/apply/bind, Prototypes & Classes

30 Questions (Easy → Moderate → Hard)

EASY LEVEL (Q1–Q10)

These questions are focused on understanding concepts.

Question 1 — Method and this

```
const user= {  
  name:"Ritik",  
  greet() {  
    console.log(this.name);  
  }  
};  
user.greet();
```

Expected Output

Ritik

What is this question asking?

Identify what this refers to when a method is called using an object.

Concepts Tested

- this
- Method Calls

Question 2 — Default Binding

```
function show() {  
  console.log(this);  
}  
show();
```

Expected Output

Depends on:

- Browser
- Node.js
- Strict Mode

Concepts Tested

- Default Binding
- this

Question 3 — Object Method

```
const car= {  
  brand:"BMW",  
  showBrand() {  
    console.log(this.brand);  
  }  
};
```

Predict output.

Concepts Tested

- this
- Object Methods

Question 4 — call()

```
function greet() {
```

```
console.log(this.name);  
}
```

```
const user= {  
  name:"Aman"  
};
```

Use call() to print:

Aman

Concepts Tested

- call()

Question 5 — apply()

```
function introduce(city) {  
  console.log(`${this.name} from${city}`);  
}
```

```
const person= {  
  name:"Ritik"  
};
```

Use apply().

Expected Output

Ritik from Bhopal

Concepts Tested

- apply()

Question 6 — bind()

```
function greet() {  
  console.log(this.name);  
}
```

```
const user= {
```

```
name:"Priya"
```

```
};
```

Create a permanently bound function.

Concepts Tested

- `bind()`

Question 7 — Arrow Function this

```
const user= {
```

```
name:"Ritik",
```

```
greet: () => {
```

```
console.log(this.name);
```

```
}
```

```
};
```

```
user.greet();
```

Predict output.

Concepts Tested

- Arrow Functions
- Lexical this

Question 8 — Event Handler Theory

What is the difference between:

```
button.addEventListener("click",function(){})
```

and

```
button.addEventListener("click", () => {})
```

regarding this?

Concepts Tested

- Event Handlers

- this

Question 9 — Constructor Function

Create:

```
function Person(name) {  
  this.name=name;  
}
```

Create an object:

```
new Person("Ritik")
```

Expected Output

```
{  
  name:"Ritik"  
}
```

Concepts Tested

- Constructor Functions

Question 10 — Prototype Lookup

```
const arr= [1,2,3];
```

Where does:

```
arr.push()
```

come from?

Concepts Tested

- Prototype Chain

MODERATE LEVEL (Q11–Q20)

Question 11 — Lost this

```
const user= {  
  name:"Ritik",  
  greet() {  
    console.log(this.name);  
  }  
};  
  
const fn =  
  user.greet;  
fn();  
Fix it.
```

Expected Output

Ritik

Concepts Tested

- bind()

Question 12 — Borrow Method Using call()

```
const user1= {  
  name:"Ritik"  
};  
  
const user2= {  
  name:"Aman"  
};
```

Create one method and use it for both objects.

Concepts Tested

- call()

Question 13 — Apply with Math.max

Find maximum number using:

Math.max()

and:

apply()

Array:

[10,20,50,5]

Expected Output

50

Concepts Tested

- apply()

Question 14 — Create Prototype Inheritance

```
const animal= {
```

```
  eats:true
```

```
};
```

Create:

dog

using:

Object.create()

Concepts Tested

- Object.create()

Question 15 — Shared Prototype Method

Create:

Person.prototype.greet

and make multiple objects use it.

Concepts Tested

- Prototype
- Memory Optimization

Question 16 — Check Prototype Relationship

```
function Person() {}
```

```
const p = new Person();
```

Verify:

```
p.__proto__ === Person.prototype
```

Expected Output

true

Concepts Tested

- **proto**
- prototype

Question 17 — Student Class

Create:

```
class Student
```

with:

```
name
```

```
marks
```

Concepts Tested

- Classes
- Constructor

Question 18 — Car Class

Create:

class Car

with:

brand

start()

Output:

BMW started

Concepts Tested

- Classes
- Methods

Question 19 — Getter

Create:

fullName

using a getter.

Example

p.fullName

should return:

Ritik Rajput

Concepts Tested

- Getters

Question 20 — Setter

Create:

fullName

setter.

Input:

p.fullName="Aman Gupta"

Expected Behavior

Update:

firstName

lastName

Concepts Tested

- Setters

HARD LEVEL (Q21–Q30)

Question 21 — Employee Inheritance

Create:

class Employee

and

class Developer

using:

extends

Concepts Tested

- Inheritance

Question 22 — Animal Hierarchy

Create:

Animal

↓

Dog

↓

Labrador

Scenario

Multi-level inheritance.

Concepts Tested

- Prototype Chain
- Classes

Question 23 — Static Method

Create:

```
MathHelper.add()
```

Example

```
MathHelper.add(5,10)
```

Output:

15

Concepts Tested

- Static Methods

Question 24 — Static Property

Create:

```
MathHelper.PI
```

Output:

3.14159

Concepts Tested

- Static Properties

Question 25 — Bank Account

Create:

```
class BankAccount
```

with:

#balance

Methods:

deposit()

withdraw()

Expected Output

700

after:

deposit(1000)

withdraw(300)

Concepts Tested

- Private Fields

Question 26 — Shopping Cart Class

Create:

class ShoppingCart

Methods:

addItem()

removeItem()

getTotal()

Scenario

E-commerce Application.

Concepts Tested

- Classes
- Arrays
- Objects

Question 27 — Library Management System

Create:

Book

Library

classes.

Methods:

addBook()

borrowBook()

returnBook()

Concepts Tested

- Classes
- OOP Design

Question 28 — Prototype vs Class Investigation

Create:

```
class Person {}
```

Verify:

```
typeof Person
```

Expected Output

```
function
```

What is this question asking?

Prove that classes are syntactic sugar.

Concepts Tested

- Classes

- Prototypes

Question 29 — Custom Array Method

Add:

`Array.prototype.last`

Example:

`[1,2,3].last()`

Output:

3

Concepts Tested

- Prototype Extension

Question 30 — Complete School Management System (Interview-Level)

Create:

`class Student`

`class Teacher`

`class School`

Requirements:

Student

name

marks

Teacher

name

subject

School

Methods:

addStudent()

addTeacher()

getTopper()

getAverageMarks()

Example Output

Topper:Ritik

AverageMarks:82

What is this question asking?

This is a mini project combining:

- this
- Classes
- Constructors
- Methods
- Arrays
- Objects
- Inheritance (optional)
- Real-world Design